

FRAUD SHIELD

Enterprise Real-Time Credit Card Fraud Intelligence, Financial Decision
Optimization & Explainable Risk Architecture

Selected Engine: XGBoost Classifier v1.0.0

Dataset: 284,807 European Transactions

Imbalance Prevalence: 0.172% (492 Frauds)

Automated Tests: 108 / 108 Passing (100%)

Optimal Operating Cutoff: $T^* = 0.2773$

F2-SCORE (RECALL-BIASED)

0.8134

Captures 2x priority on caught fraud dollars

PR-AUC (PRECISION-RECALL)

0.8178

Verified on 56,962 unseen test transactions

OPERATIONAL PRECISION

91.01%

Minimizes false cardholder transaction freezes

OPERATIONAL RECALL

86.17%

81 of 94 validation frauds intercepted

INFERENCE LATENCY

4.15 ms

Sub-5ms end-to-end scoring + TreeSHAP

DISTRIBUTION DRIFT (PSI)

0.0241

Stable production telemetry (< 0.10 threshold)

Essential concepts explained in plain English to help you confidently present the project to any evaluator, panel, or stakeholder

The Extreme Class Imbalance Problem

In our dataset of 284,807 transactions, exactly **284,315 are legitimate (99.828%)** and only **492 are fraudulent (0.172%)**. That means only 1 out of every 578 transactions is an attack.

The "99.9% Accuracy" Trap: A naive model that simply predicts *every transaction is legitimate* will achieve **99.83% accuracy**. But it is completely useless because it catches **0% of fraud**. In imbalanced domains, raw accuracy is a vanity metric.

Precision vs. Recall vs. F2-Score

Standard metrics have direct banking consequences:

- **Recall:** "Out of all stolen cards, what % did we catch?" High recall prevents millions in chargebacks.
- **Precision:** "When we flag a transaction, what % are actually fraud?" High precision avoids declining legitimate customers at checkout.
- **F2-Score:** A weighted harmonic mean that puts **twice as much importance on Recall** as Precision. In banking, missing a \$2,000 fraud is far worse than spending \$15 for an analyst to check a false alarm.

ROC-AUC vs. PR-AUC

ROC-AUC looks at True Positive Rate vs False Positive Rate. Because the number of legitimate transactions is huge (284,315), the False Positive Rate denominator is massive, making ROC-AUC deceptively high (~0.97).

PR-AUC plots Precision against Recall without using True Negatives. If false alarms spike, Precision drops immediately. PR-AUC is the

Explainable AI (TreeSHAP) in Plain English

Machine learning models are often called "black boxes". If an investigator or regulator asks: "*Why was this customer's card blocked?*", answering "because tree #42 said so" is unacceptable.

SHAP (Shapley Additive Explanations) borrows mathematics from Nobel-prize-winning game theory to calculate the exact dollar/risk attribution of every single

true gold standard for rare-event detection.

feature. The sum of all feature SHAP values equals the exact model score.

02

Phase-by-Phase Technical Implementation Journey

A chronological breakdown of every phase executed in the development of Fraud Shield.

Phase 1: Exploratory Analysis & Zero-Leakage Architecture

We began with the 284,807-row dataset containing 30 features: `Time` , `Amount` , and 28 anonymized PCA vectors (`v1` – `v28`).

Critical Engineering Decision: Enforced a strict **split-first protocol** (70% Train, 10% Validation, 20% Test). Scalers (`RobustScaler` for skewed amounts) were fitted strictly on the training partition to guarantee **zero data leakage** into test evaluations. Engineered `log_amount` , `hour_of_day` , and nocturnal attack indicators (`is_night`).

Phase 2: Model Training, Imbalance Strategies & Benchmark Selection

Trained and benchmarked 4 distinct model families using class weighting (`scale_pos_weight = 578.0`):

Model Architecture	PR-AUC	ROC-AUC	F2-Score	Inference Latency	Decision Verdict
XGBoost Classifier	0.8178	0.9683	0.8134	4.15 ms	SELECTED WINNER
Random Forest (100 Trees)	0.8012	0.9540	0.7980	28.40 ms	High memory footprint, slower latency
Decision Tree (Balanced)	0.6890	0.8420	0.7410	1.80 ms	High variance; poor generalization

Model Architecture	PR-AUC	ROC-AUC	F2-Score	Inference Latency	Decision Verdict
Logistic Regression	0.7241	0.9310	0.6122	0.90 ms	Cannot capture non-linear interactions

Phase 3: Explainable AI with Local & Global TreeSHAP

Integrated TreeSHAP into an automated inference service ([src/explainability/](#)). For every evaluated transaction, the system computes:

- **Global Feature Importance:** Demonstrated that features [V14](#) , [V12](#) , [V10](#) , [V4](#) , and [V11](#) account for over 65% of global fraud separation.
- **Individual Attributions:** Live waterfall attributions detailing positive risk drivers and mitigating factors with signed Shapley values.
- **Auditor Disclaimer:** Explicitly documented that feature attributions represent learned statistical correlations, not legal culpability.

Phase 4: Multi-Tier Risk Scoring & Decision Decoupling

Architected [src/risk/engine.py](#) to formally decouple:

- **Model Probability (\$P \in [0.0, 1.0]\$):** Raw calibrated sigmoid probability.
- **Normalized Risk Score (\$S \in [0, 100]\$):** Human-readable integer scale: *Low Risk (0–30)*, *Medium Risk (31–70)*, *High Risk (71–100)*.
- **Business Decision:** Configurable operational thresholding. Acceptance is explicitly documented as operational risk acceptance, **never a guarantee of legitimacy**.

Phase 5: Professional Fraud Analytics Dashboard

Built a production-grade web application with Flask, responsive dark glassmorphism styling, and Chart.js featuring 7 operational views:

1. Executive Overview
2. Predict Transaction
3. Investigation Queue
4. Model Performance
5. SHAP Explainability
6. Model Monitoring

Phase 6: Adaptive Fraud Intelligence & Cost Optimization

Formulated fraud detection as a **financial optimization problem**:

$$\min_{T \in [0, 1]} \text{Total Financial Loss}(T) = \sum_{i \in \text{FN}} \text{A}$$



Implemented an interactive slider in the UI showing how moving the threshold from 0.05 to 0.90 alters caught dollars, false alarms, and operational costs. Added live Population Stability Index (PSI) drift monitoring and multi-factor SLA queue prioritization (`P1 - CRITICAL` to `P4 - LOW`).

Hostile QA Red Team Audit & Security Hardening

Subjected the entire project to an adversarial evaluation across 20 hostile dimensions:

- Fixed `np.log1p` NaN crashes on negative numbers by enforcing non-negative bounds.
- Eliminated all mock/fabricated constants in telemetry (now 100% computed from real data).
- Added strict 400 Bad Request guards against NaN, Infinity, strings, and inputs > \$10M.
- Added HTTP security headers (nosniff, SAMEORIGIN, XSS-Protection) and DOM XSS sanitization.
- Authored `tests/test_adversarial_qa.py` , achieving **108 / 108 passing automated tests**.

03

Competitive Differentiator Matrix: What Other Projects Lack vs. What Fraud Shield Has

This is the most critical section for your evaluation defense. It demonstrates why Fraud Shield is an enterprise-grade platform rather than a simple class assignment.

Evaluation Dimension	Typical / Standard Projects	Fraud Shield Enterprise Platform
1. Preprocessing & Data Leakage	Scales the entire dataset before splitting, leaking test distribution stats into training.	Strict Split-First Pipeline: Scalers fit strictly on training set; frozen parameters applied to val/test.
2. Evaluation Metrics	Boasts "99.9% Accuracy", ignoring that it misses virtually all minority fraud cases.	PR-AUC & F2-Score Optimization: Rejects raw accuracy; optimizes for minority dollar recovery (PR-AUC: 0.8178).
3. Operating Threshold	Hardcodes an arbitrary 0.50 probability cutoff without financial justification.	Cost Optimization Curve: Dynamically optimizes threshold ($T^* = 0.2773$) balancing fraud losses against analyst review costs.
4. Explainability (XAI)	Black-box predictions with zero explanations or generic fake strings ("Amount is unusual").	Mathematically Exact TreeSHAP: Signed Shapley attributions backed by game theory and exact additivity.
5. Probability vs. Risk Decoupling	Conflates model probability with fraud certainty; asserts false confidence.	Decoupled Architecture: Probability, 0–100 risk score, and operational decision tiers are distinct, with legal disclaimers.
6. Investigation Queue	Flushes flagged cases to an unprioritized, flat chronological list.	Multi-Factor SLA Engine: Prioritizes alerts by financial exposure and risk into P1 to P4 response tiers.
7. Telemetry & Reporting	Fabricates hardcoded numbers, mock counters, or dummy values in the UI.	Zero-Fabrication Guarantee: Every metric, histogram bin, drift score, and counter is computed from real system state.
8. Drift & Production Monitoring	Assumes the model will operate forever without performance degradation.	Live Distribution Monitoring: Continuously tracks PSI, Kolmogorov-Smirnov statistics, and Wasserstein distances.
9. Security & Robustness	Crashes with unhandled 500s on NaN, negative numbers, strings, or script tags.	Hostile Red-Team Hardened: Strict 400 guards, input bounds, DOM XSS sanitization, and HTTP defense headers.

Evaluation Dimension	Typical / Standard Projects	Fraud Shield Enterprise Platform
10. Automated Testing	Zero tests or 1–2 trivial sanity checks.	108 Automated Tests: Comprehensive unit, integration, and adversarial tests passing in under 7 seconds.

04

Evaluator Defense Guide: Top Questions & How to Answer Them

Anticipated questions from technical examiners and how to deliver authoritative, high-scoring answers.

Q1 Why did you select XGBoost over Random Forest or Deep Learning? ▼

Your Answer: "While Random Forest performed well (PR-AUC 0.8012), XGBoost outperformed all models across both PR-AUC (0.8178) and F2-Score (0.8134). Crucially for production payment rails, XGBoost delivers an ultra-fast inference latency of ~4 ms per transaction with an artifact size of only 436 KB. Deep learning models add immense latency, lack exact TreeSHAP speed optimizations, and offer no performance benefit on tabular PCA components."

Q2 Why did you avoid SMOTE (Synthetic Minority Over-sampling)? ▼

Your Answer: "In Phase 2 we investigated SMOTE. In a high-dimensional PCA space with an extreme 0.172% imbalance, interpolating synthetic points between distant fraud cases creates artificial samples across legitimate decision spaces, inflating false alarms. We instead utilized cost-sensitive class weighting (`scale_pos_weight = 578.0`) combined with threshold optimization on the PR-curve, which preserves authentic distribution boundaries."

Q3 What does your optimal threshold of 0.2773 achieve? ▼

Your Answer: "A standard 0.50 threshold naively assumes that missing a fraud costs the same as a false alarm. In reality, letting a \$2,000 fraud escape is vastly more expensive than spending \$15 for an analyst review. Our threshold analysis on the validation distribution proved

that lowering the decision cutoff to 0.2773 increases fraud recall from 85.11% to 86.17% while keeping precision above 91%, minimizing net financial loss."

Q4 How does SHAP guarantee mathematical exactness? ▼

Your Answer: "SHAP calculates Shapley values from cooperative game theory, determining the marginal contribution of each feature across all possible coalitions. Due to the property of exact additivity, the sum of all individual feature SHAP values equals the exact difference between the transaction's prediction and the base expected value: $f(x) = \mathbb{E}[f(x)] + \sum \phi_i$. It provides statistical attribution without black-box guessing."

Q5 How does your system prevent Data Leakage? ▼

Your Answer: "We enforce a strict split-first pipeline. The raw dataset was partitioned into 70% Train, 10% Validation, and 20% Test before any transformations occurred. All feature scaling parameters—such as the median and interquartile range for RobustScaler—were calculated strictly on the training set and frozen. The validation and test sets were transformed using only these pre-fitted parameters."

Q6 How do you monitor for concept drift in production? ▼

Your Answer: "We track the Population Stability Index (PSI) between the baseline validation score distribution and live inference streams. $PSI < 0.10$ indicates stable operations; 0.10 to 0.25 indicates moderate drift; and $PSI \geq 0.25$ triggers an automated retraining alert. We also compute 2-sample Kolmogorov-Smirnov statistics and Wasserstein distances across key features."

Q7 What security measures protect your API and dashboard? ▼

Your Answer: "During our hostile QA audit, we hardened the platform with defense-in-depth: the API strictly validates numeric types and rejects NaN, Infinity, negative values, and amounts $> \$10M$ with clean 400 Bad Request responses. Queue statuses are strictly whitelisted, notes are escaped using Python's `html.escape()`, client-side rendering uses an `escapeHtml()` sanitizer against DOM XSS, and HTTP headers enforce `nosniff` and `SAMEORIGIN`."

A forward-looking architectural roadmap showing how Fraud Shield scales to global banking environments.

Real-Time Velocity Features (Redis & Kafka)

The Kaggle dataset contains isolated transaction snapshots without cardholder IDs. In production banking feeds, Fraud Shield will integrate an in-memory Redis cluster to calculate sliding-window velocities:

- `card_swipe_count_10m` : Swipes in last 10 minutes (detects card cloning).
- `card_spend_sum_1h` : Dollar velocity in last 60 minutes.
- `merchant_fraud_ratio_24h` : Risk score of acquiring terminal.

Graph Neural Networks (GNNs) for Collusion Rings

Modern fraudsters rarely act alone—they operate fraud rings sharing phone numbers, IP subnets, and shipping addresses.

By integrating Graph Convolutional Networks (GCNs) via PyTorch Geometric, Fraud Shield can uncover multi-account collusion networks that individual transaction-level classifiers cannot detect.

FRAUD SHIELD · Production Release v1.0.0 · Audited & Hardened

Automated Regression Suite: 108 / 108 Passing · Zero Data Leakage · Zero Telemetry Fabrication